
On-Device Diffusion

Mahad Faruqi
Purdue EcoAI Lab
Advisor: Dr. Haoran You

1 Motivation

Deploying diffusion models on resource-constrained devices requires balancing generation latency, memory usage, and image quality. Recent work on consumer-GPU diffusion demonstrates that text-encoder size, numerical precision, and weight residency all influence this tradeoff [8]. In FLUX, a generation repeatedly invokes a denoising transformer, while the surrounding pipeline also performs text encoding and image decoding [3, 11]. Efficient deployment therefore requires considering the complete pipeline and the resources available on the target device. Acceptable quality loss will be treated as a deployment requirement: an interactive phone application may prioritize responsiveness and fitting in memory, while a fidelity-sensitive workflow may tolerate longer generation times. The acceptable tradeoff depends on the intended use and user preference.

Hardware-specific implementations can make a substantial difference. SVDQuant combines low-bit weights and activations with specialized fused kernels to reduce diffusion inference memory and latency [15]. More broadly, ML Drift studies execution of large generative models across mobile and desktop GPUs, addressing differences in GPU interfaces and execution strategies [25]. Memory strategies also depend on the platform: the edge-dit.cpp runtime explicitly avoids redundant CPU-to-GPU weight staging on Apple unified-memory devices, where staging can introduce additional copies within the same physical memory pool [27]. These examples motivate measuring optimization choices on each target rather than assuming that a configuration transfers efficiently between devices.

Existing systems provide both deployment paths and strong comparison points. Stable-diffusion.cpp supports quantized diffusion inference across multiple hardware backends, while edge-dit.cpp includes automatic quantization and placement under a memory budget [24, 26]. Compiler-based deployment also has precedents: MLC Web Stable Diffusion uses TVM for model compilation and local execution, and AMD-SHARK provides FLUX model recipes and serving applications built around IREE [18, 1]. The research question is therefore how a shared compiler/runtime path can achieve efficient execution under different device constraints, and which target-specific optimizations measurably improve it.

This thesis will establish a shared deployment workflow for FLUX image generation and Wan video generation, beginning with image inference and benchmarking both workloads against existing runtimes on resource-constrained hardware. Here, a generic path means a shared model implementation and compilation workflow, with target-specific code generation and runtime support. The evaluation will measure end-to-end and per-stage latency, peak memory, and output quality for each workload. Profiling will then guide a focused optimization in numerical precision, kernel execution, or memory management. Cross-step reuse remains a candidate technique. Prior work studies approximate feature reuse and its interaction with quantization [16, 17]. Qwen-Image 2.1 provides a complementary example: its attention and conditioning design allows text and reference-image keys and values to be cached across denoising steps without approximating changing features [13, 23]. Applicability will depend on the selected model architecture and measured workload. The goal is to determine how far the generic path can be improved and when hardware-specific specialization is necessary for practical on-device execution.

2 Target Models and Backends

The target workloads are text-to-image generation with FLUX.2 [klein] 4B and text-to-video generation with Wan 2.1 T2V-1.3B. FLUX provides a distilled image model with a four-step reference configuration [4]. Wan 2.1 T2V-1.3B provides the initial video workload, starting with short clips at its recommended 480p resolution [28]. Both workloads will include text encoding, denoising and image or video decoding. Video experiments will record frame count, frame rate and resolution alongside the sampling settings.

We will establish image inference first, then video inference, and test whether the chosen optimization transfers between them. Quantized and distilled variants will be evaluated with their quality tradeoffs recorded separately. FastVideo provides a video inference framework and accelerated Wan variants that we can benchmark and reuse where compatible [10]. Its Apple deployment path uses MLX and dedicated FastMetal checkpoints, so CUDA and Apple implementations will be checked separately [7]. The undistilled FLUX.2 [klein] 4B Base remains an optional workload for longer denoising runs [5]. Each model will have its own reference outputs, and changes to checkpoints will be distinguished from execution-only optimizations.

We will target the NVIDIA Jetson Orin Nano through CUDA for embedded deployment and the M1 MacBook Pro through Metal for personal-device deployment. The Jetson provides configurable power limits [21], while the NVIDIA A100 will serve as a CUDA development and performance reference [19]. Together, these devices allow comparisons across backends and between server and embedded CUDA execution, with improvements measured against each device’s baseline. Jetson’s shared CPU–GPU memory also makes total memory usage important as CPU offloading alone does not reduce the physical memory required [20].

The shared compiler/runtime still needs to be selected. IREE is a candidate supporting both CUDA and Metal [14]. Initial checks can establish operator coverage, correctness, and the precision and memory needed for the complete pipeline. We will document device variants, available memory, software versions, and the Jetson power mode to make comparisons reproducible.

3 Pipeline Architecture

On-Device Diffusion: From Model to Deployment

PREPARATION: compile and validate before deployment

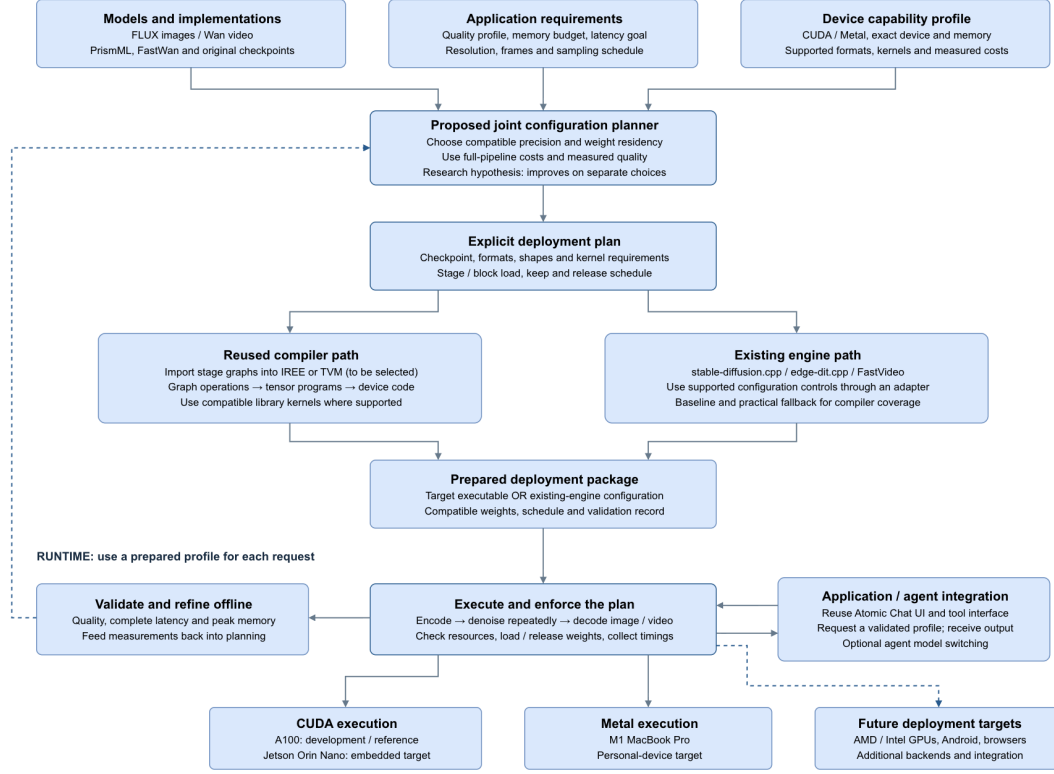


Figure 1: On-device diffusion architecture: model preparation, reused execution paths, and runtime generation across hardware targets.

Figure 1 separates model preparation from runtime execution within the proposed on-device diffusion engine. Preparation combines FLUX or Wan checkpoints, application requirements for quality, latency and memory, and the target device’s capabilities. These inputs guide the choice of precision, compatible kernels and memory settings. We will initially compare explicit configurations using existing tools and model variants, including PrismML’s Bonsai checkpoints and FastVideo’s accelerated Wan implementations [10, 22].

The selected configuration records the checkpoint, weight formats, supported input shapes and component loading settings. In the compiler path, stage graphs for text encoding, denoising and decoding are imported into a reused compiler, potentially IREE, and lowered into tensor computations and target-specific device code [14]. Compatible library kernels may implement selected operations. In the existing-engine path, an adapter applies supported settings to stable-diffusion.cpp, edge-dit.cpp or FastVideo, reusing model execution, kernels and runtime management [10, 24, 26]. These engines provide working baselines and execution options where compiler integration is incomplete. Both paths require checking weight formats, operators and interfaces.

Preparation produces a deployment package containing compatible weights, compiled modules or engine settings, and a tested configuration, referred to as a profile. Validation checks output quality, latency and peak memory for the specified workload. For example, an image profile could specify a quantized FLUX checkpoint, resolution and component loading policy, while a video profile would additionally specify frame count and frame rate. Compilation is performed where required, and prepared artifacts are reused across requests.

At runtime, an application supplies a prompt and selects a supported profile. The runtime checks request compatibility and available resources, manages buffers and component loading, and performs text encoding, repeated denoising and image or video decoding. Atomic Chat provides an application-level reuse example for request handling, progress reporting and presentation [2]. CUDA serves A100 and Jetson, while Metal serves M1. Offline measurements feed back into configuration selection, as shown by the feedback arrow. The dashed deployment branch represents future targets. Our initial work connects and evaluates these reused components through a shared workflow, with compiler or runtime extensions guided by measured limitations.

Agent integration. As an extension, a local agent could invoke the diffusion pipeline through a structured tool interface. Using device information supplied by the runtime, the agent would translate the user’s request into application requirements, such as a compact profile for a quick illustration or a fidelity-focused profile for a detailed image. The agent would select only among configurations that were validated and compiled ahead of time, so that compilation never occurs on the interactive path; the runtime then loads or reuses the corresponding executable and returns the generated image so the agent can resume its response.

On a shared-memory device, the agent’s language model and the diffusion pipeline compete for the same physical memory: the reference FLUX.2 [klein] 4B configuration alone is documented to need roughly 13 GB of GPU memory [4], while the Jetson Orin Nano has 8 GB shared between CPU and GPU [21, 20]. An image request therefore becomes a residency decision spanning both models: which of the language model’s weights, its KV cache, and each diffusion stage to keep, evict, or reload, and when. We would compare simple policies, such as fully unloading the language model or retaining only its KV cache, and examine whether buffers, or weights where architectures match, can be shared between the agent’s language model and the pipeline’s text encoder. This extends the memory planning in RQ3 from a single pipeline to the complete agent to image to agent turn, measured by end-to-end turn latency, peak memory, and the time to restore the agent’s context.

Integration with a host such as Atomic Chat would allow evaluation of complete interactions [2]. Agent-selected profiles will be compared with three baselines: always using the compact profile, a fixed rule-based mapping from request type to profile, and a profile chosen by the user. Using a small labeled prompt set with an intended profile for each request, we will measure how often each policy meets the intended quality requirement and at what latency and memory cost. Whether agent selection improves on these baselines should be tested.

We will evaluate quality, latency, and peak memory on each device. Profiling will determine which configuration, compiler, or runtime changes merit development. The architecture identifies integration boundaries and the thesis contribution will be a measured improvement beyond the reused components.

4 Implementation Approach and Candidate Optimizations

We will build on existing open-source runtimes, quantization tools, and kernels. Stable-diffusion.cpp and edge-dit.cpp provide runtime foundations to evaluate [24, 26]; PrismML’s Bonsai Image provides compressed FLUX.2 [klein] variants with CUDA and Apple deployment paths [22]. Atomic Chat’s open image-generation integration with stable-diffusion.cpp provides an application-level reference [2]. FastVideo provides an additional video baseline and a source of existing Wan acceleration methods [10]. We will verify model and device compatibility before choosing components to reuse or adapt.

After validating outputs, we will profile stage latency, allocations, and transfers, recording any changes needed to fit each device in its baseline. Profiling will guide experiments in four areas:

- **Quantization:** Compare existing 8-bit and 4-bit weight-only methods, weight-and-activation methods such as SVDQuant, and pretrained binary or ternary variants where supported [12, 15, 22]. We will reuse quantized checkpoints or prepare them with existing tools before inference. Precision may differ between pipeline components; compressed weights require suitable kernels and do not guarantee faster execution.
- **Kernel optimization:** Reuse efficient attention, fused operations, and low-bit matrix kernels. Tiled attention reduces memory traffic, while fusion can avoid intermediate buffers and

separate launches [6, 15]. We will consider custom changes only for bottlenecks left by existing implementations.

- **Memory management:** Reduce peak allocations through buffer reuse, releasing unused stage resources, and tiled decoding where supported. Placement and transfer strategies will follow each device’s memory architecture, accounting for latency costs and possible tiling artifacts [11, 20].
- **Cross-step reuse:** Distinguish exact caching of step-independent context, where the architecture permits it, from approximate reuse or prediction of changing features [13, 16]. We will prioritize the longer-step base model for approximate reuse experiments and measure cache memory, speed, and any quality loss.

Quality–resource tradeoffs. We will evaluate several quality requirements rather than impose one universal tolerance. A compact, responsive profile may accept more loss of detail or prompt fidelity. A fidelity-focused profile will require closer agreement with the reference quality. Requirements will be specified before selecting configurations, with separate prompts for configuration selection and held-out evaluation. Quality metrics and structured visual review will assess the outputs. We will report the latency–memory–quality tradeoffs and compare methods at matched quality requirements where feasible. A faster result with lower quality will be reported as a tradeoff. Metric choices and acceptance criteria will be defined in the evaluation section.

We will isolate each technique’s effect before testing combinations. The research contribution will address a measured gap in deployment or configuration selection beyond integrating existing components.

5 Research Questions and Expected Contribution

The thesis will investigate three questions:

1. **RQ1: How well do existing solutions transfer across devices?** Using the same model and workload where supported, we will compare execution on the A100, Jetson Orin Nano, and M1 MacBook Pro. We will identify compatibility gaps, changes required to fit each device, and whether configurations that perform well on one target remain effective on another. Comparisons with tuned baselines on each device will distinguish configuration limitations from hardware differences.
2. **RQ2: How do quality requirements change the preferred configuration?** We will examine how precision, kernel choices, and memory policies affect generation latency, peak memory, and output quality. For each device and predefined quality requirement, we will identify the fastest evaluated configuration that fits the memory budget. Separate selection and evaluation prompts will test whether these choices generalize beyond the tuning workload.
3. **RQ3: What compiler or runtime improvement addresses a measured gap?** Profiling will identify a bottleneck that persists after tuning existing options. We will develop one focused improvement, such as a buffer-lifetime optimization or a better execution-partitioning policy, and isolate its effect through controlled comparisons under the same quality and resource requirements.

The engineering foundation comprises a reproducible deployment and benchmarking workflow built from existing models, runtimes, quantization tools, and kernels. The expected research contribution is an evidence-based account of which configurations transfer across targets and quality requirements, together with a focused compiler or runtime improvement that addresses an identified limitation. Its novelty will be assessed against prior work and experiments will establish its benefits, costs, and limits of applicability.

6 Evaluation Plan

Baselines and workloads. We will compare the shared deployment path with supported stable-diffusion.cpp and edge-dit.cpp configurations, including their automatic fitting options and documented tuning [24, 26]. Video comparisons will include the Wan reference implementation and

compatible FastVideo configurations [28, 10]. Each device will have its own baseline, and the A100 will also run reference implementations for quality comparisons. Experiments will fix the model, prompts, resolution, sampling schedule, guidance settings and, for video, frame count and frame rate, with multiple seeds and matched initial noise where supported. Compressed checkpoints will be reported separately from execution-only changes.

Latency and memory. We will measure complete generation latency and time spent in text encoding, denoising, and decoding. Video timings will cover complete clip generation. Cold-start time, including model loading and any compilation, will be reported separately from repeated generation with the model loaded. Timings will wait for device work to finish and report medians and variability across repeated runs. Peak memory measurements will cover the complete pipeline, distinguish runtime allocations from observed device or system usage, and avoid double-counting shared CPU–GPU memory.

Quality requirements. For images, we will assess object-level prompt adherence using GenEval [9], supplemented by blinded paired visual review of detail, artifacts, and prompt fidelity on a fixed broader prompt set. Video review will additionally assess temporal consistency, motion and flicker using a predefined rubric on held-out video prompts.

Testing the contribution. For RQ1, we will transfer supported configurations between devices and compare them with configurations tuned locally. For RQ2, we will compare the fastest evaluated configurations satisfying each quality and memory requirement. For RQ3, we will enable and disable the proposed improvement within the same stack, holding other settings fixed, then compare against tuned external baselines. These experiments will isolate its effect and identify workloads where its overhead outweighs its benefits.

7 Milestones and Feasibility

Implementation will proceed through four milestones, with feasibility checks before committing to substantial optimization work.

1. **Establish baselines and choose a direction.** Begin with inference on the A100 and Jetson using existing software. Identify initial bottlenecks, resolve design-doc comments and choose a provisional research direction in week 1. Add M1 measurements and check prior work before confirming the hypothesis in week 3. Deliverables are baseline results and an updated proposal.
2. **Build the shared deployment path.** Select the compiler/runtime foundation, integrate the denoiser first, and connect text encoding and decoding. Validate outputs before tuning performance. Deliverables are an executable pipeline, documented backend adapters and fallbacks, and initial end-to-end and per-stage measurements on supported targets.
3. **Implement and test one focused improvement.** Build the improvement selected from the early measurements and prior-work review. Compare it with tuned baselines under the predefined quality requirements. Deliverables are a switchable prototype and controlled comparisons.
4. **Complete evaluation and reporting.** Freeze configurations, run held-out quality tests and repeated performance measurements, and compare against tuned baselines. Deliverables are experiment scripts, configuration records, results tables, and the thesis analysis of benefits, overheads, and limitations.

Feasibility and scope. On Jetson, we will establish a working FLUX configuration using supported compressed checkpoints and memory-management techniques, then measure its quality, latency, and peak memory. Quantization, sequential stage loading, and tiled decoding will be considered deployment options throughout the study. Changes to resolution or other workload settings will be reported explicitly so that comparisons remain meaningful.

If the selected compiler lacks required operators or weight-format support, we will investigate supported decompositions and compatible library calls. Where necessary, affected stages will remain in an existing runtime, with integration overhead measured. A working runtime path will support

continued experiments while we assess which compiler extensions are justified by the measured limitations. Additional models, deployment platforms, and agent integration will remain extensions until the core FLUX and Wan experiments are complete.

Weekly implementation schedule.

Week / dates	Goal	Deliverable	Level
1: Sep 28–Oct 2	Run inference on Jetson and A100 with an existing runtime. Identify initial bottlenecks, resolve design-doc comments and choose a provisional research direction.	Initial results and revised proposal	3/5
2: Oct 5–9	Add M1 image inference and establish a Wan video baseline on A100. Repeat device measurements. Compare supported configurations and define quality criteria. Check the proposed direction against prior work.	Baseline comparison and quality criteria	3/5
3: Oct 12–16	Test denoiser compilation and target support. Choose the compiler or runtime foundation and define one testable improvement from the measured gap.	Chosen stack and research hypothesis	4/5
4: Oct 19–23	Connect encoding, denoising and decoding in the selected stack. Check correctness and memory use on the supported targets.	Working pipeline and benchmark scripts	5/5
5: Oct 26–30	Implement the chosen optimization on one device. Make it optional so its effect can be measured against the unchanged pipeline.	First optimization prototype	5/5
6: Nov 2–6	Compare the prototype with tuned baselines under the same quality requirements. Fix regressions and investigate when the improvement helps.	Refined prototype and initial comparison	4/5
7: Nov 9–13	Evaluate the improvement across FLUX and Wan on supported targets. Record benefits, overheads and limitations. Confirm the scope of the final experiments.	Cross-device results	4/5
8: Nov 16–20	If core results are ready, connect one local agent to image generation on one device. Measure model switching and response resumption. Otherwise, finish core experiments.	Agent demo or completed core experiments	4/5
9: Nov 23–27	Resolve remaining issues, organize logs and figures, and freeze configurations for final testing.	Frozen configurations and draft figures	2/5
10: Nov 30–Dec 4	Run held-out quality tests and repeated timings. Compare the improvement enabled and disabled. Complete baseline comparisons and analyze variability.	Final results and comparison tables	4/5
11: Dec 7–11	Finish the report, reproduce key results and prepare the presentation and demonstration.	Report, code and presentation	3/5

The provisional direction will be chosen at the end of week 1 and refined through baseline measurements and a prior-work check. By the end of week 3, we will commit to one research hypothesis and a feasible compiler/runtime path. If compilation remains blocked, we will develop the improvement in an existing runtime. Initial Jetson experiments will establish a working model and quantization configuration, recording the resulting memory use, latency, and output-quality tradeoffs.

Agent integration in week 8 depends on core results being ready after week 7. It will use validated configurations compiled in advance where required. If pursued, profile-selection comparisons will follow the evaluation described above, with held-out agent requests included in week 10.

References

- [1] AMD-SHARK contributors. AMD-SHARK: Inference modeling and serving. GitHub repository. URL <https://github.com/nod-ai/amd-shark-ai>. Accessed September 21, 2026.
- [2] AtomicBot-ai contributors. Atomic Chat: Image generation with stable-diffusion.cpp. GitHub pull request 300, 2026. URL <https://github.com/AtomicBot-ai/Atomic-Chat/pull/300>. Open pull request; accessed September 21, 2026.
- [3] Black Forest Labs. FLUX reference implementation: Sampling. GitHub source code. URL <https://github.com/black-forest-labs/flux/blob/main/src/flux/sampling.py>. Accessed September 21, 2026.
- [4] Black Forest Labs. FLUX.2 [klein] 4B: Model card. Hugging Face model documentation, 2026. URL <https://huggingface.co/black-forest-labs/FLUX.2-klein-4B>. Accessed September 21, 2026.
- [5] Black Forest Labs. FLUX.2 [klein] 4B Base: Model card. Hugging Face model documentation, 2026. URL <https://huggingface.co/black-forest-labs/FLUX.2-klein-base-4B>. Accessed September 21, 2026.
- [6] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness, 2022. URL <https://arxiv.org/abs/2205.14135>.
- [7] FastVideo contributors. FastVideo documentation: Apple silicon FastWan. Online documentation. URL https://haoailab.com/FastVideo/getting_started/installation/mps/. Accessed September 23, 2026.
- [8] F. Ganz and M. Müller. The weight is over: Interactive diffusion on consumer GPUs, 2026. URL <https://arxiv.org/abs/2609.21849>. arXiv version 1, September 18, 2026.
- [9] D. Ghosh, H. Hajishirzi, and L. Schmidt. GenEval: An object-focused framework for evaluating text-to-image alignment, 2023. URL <https://arxiv.org/abs/2310.11513>.
- [10] Hao AI Lab and FastVideo contributors. FastVideo: A unified inference and post-training framework for accelerated video generation. GitHub repository. URL <https://github.com/hao-ai-lab/FastVideo>. Accessed September 23, 2026.
- [11] Hugging Face. Diffusers documentation: Reduce memory usage. Online documentation, . URL <https://huggingface.co/docs/diffusers/optimization/memory>. Accessed September 21, 2026.
- [12] Hugging Face. Diffusers documentation: Quantization. Online documentation, . URL <https://huggingface.co/docs/diffusers/quantization/overview>. Accessed September 21, 2026.
- [13] Hugging Face. Diffusers documentation: QwenImage21Transformer2DModel. Online documentation, . URL https://huggingface.co/docs/diffusers/main/api/models/qwenimage21_transformer2d. Development version; accessed September 22, 2026.
- [14] IREE contributors. IREE documentation: Deployment configurations. Online documentation. URL <https://iree.dev/guides/deployment-configurations/>. Accessed September 21, 2026.
- [15] M. Li, Y. Lin, Z. Zhang, T. Cai, X. Li, J. Guo, E. Xie, C. Meng, J.-Y. Zhu, and S. Han. SVDQuant: Absorbing outliers by low-rank components for 4-bit diffusion models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2411.05007>.
- [16] J. Liu, C. Zou, Y. Lyu, J. Chen, and L. Zhang. From reusing to forecasting: Accelerating diffusion models with TaylorSeers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 15853–15863, 2025. URL https://openaccess.thecvf.com/content/ICCV2025/html/Liu_From_Reusing_to_Forecasting_Accelerating_Diffusion_Models_with_TaylorSeers_ICCV_2025_paper.html.

- [17] X. Liu, Z. Li, and Q. Gu. CacheQuant: Comprehensively accelerated diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 23269–23280, 2025. URL https://openaccess.thecvf.com/content/CVPR2025/html/Liu_CacheQuant_Comprehensively_Accelerated_Diffusion_Models_CVPR_2025_paper.html.
- [18] MLC AI contributors. Web Stable Diffusion: Bringing stable diffusion models to web browsers. GitHub repository. URL <https://github.com/mlc-ai/web-stable-diffusion>. Accessed September 21, 2026.
- [19] NVIDIA. NVIDIA A100 Tensor Core GPU. Product documentation, . URL <https://www.nvidia.com/en-us/data-center/a100/>. Accessed September 21, 2026.
- [20] NVIDIA. CUDA for Tegra: Memory management and porting considerations. Online documentation, . URL <https://docs.nvidia.com/cuda/cuda-for-tegra-appnote/index.html>. Accessed September 21, 2026.
- [21] NVIDIA. Jetson Orin Nano Developer Kit: User guide. Online documentation, . URL <https://docs.nvidia.com/jetson/orin-nano-devkit/user-guide/latest/>. Accessed September 21, 2026.
- [22] Prism ML. Bonsai Image Ternary 4B: Model card and deployment details. Hugging Face model documentation, 2026. URL <https://huggingface.co/prism-ml/bonsai-image-ternary-4B-gemlite-2bit>. Accessed September 21, 2026.
- [23] Qwen Team. Qwen-Image-2.1: Compact, efficient, and unified image creation. Official release blog, Sept. 2026. URL <https://qwen.ai/blog?id=qwen-image-2.1>. Published September 20, 2026; accessed September 22, 2026.
- [24] stable-diffusion.cpp contributors. stable-diffusion.cpp: Diffusion model inference in C/C++. GitHub repository. URL <https://github.com/leejet/stable-diffusion.cpp>. Accessed September 21, 2026.
- [25] J. Tang and I. for Large Generative Models, 2025. URL <https://arxiv.org/abs/2505.00232>.
- [26] THU-MIG. edge-dit.cpp: A native C/C++ inference engine for diffusion transformers. GitHub repository, . URL <https://github.com/THU-MIG/edge-dit.cpp>. Accessed September 21, 2026.
- [27] THU-MIG. edge-dit.cpp: Memory-efficient execution. Online documentation, . URL <https://github.com/THU-MIG/edge-dit.cpp/blob/main/docs/optimization/memory-efficient-execution.md>. Accessed September 21, 2026.
- [28] Wan Team. Wan2.1-T2V-1.3B: Model card and reference implementation. Hugging Face model documentation, 2025. URL <https://huggingface.co/Wan-AI/Wan2.1-T2V-1.3B>. Accessed September 23, 2026.